UDF
Centro Universitário

ALGORITMOS E
PENSAMENTO COMPUTACIONAL
2026.2

UNIDADE I

Fundamentos da Introdução à Programação e ao Pensamento Computacional em C

Prof. Me. Gleidson Porto Batista

UDF – Centro Universitário

ALGORITMO LÓGICA PROBLEMAS MODELAGEM PROGRAMAÇÃO SOLUÇÕES







Seu futuro,
nossa inspiração.

CIÊNCIA DA COMPUTAÇÃO
SISTEMAS DE INFORMAÇÃO | ENGENHARIA DE SOFTWARE

A Unidade I consolida a transição da lógica abstrata para a programação estruturada

1. Compreensão de Algoritmos

Dominar o conceito de algoritmo como sequência finita, ordenada e não ambígua de passos para resolver problemas complexos.

2. Pensamento Computacional

Aplicar os 4 pilares fundamentais (Decomposição, Padrões, Abstração e Algoritmos) na estruturação de soluções automatizáveis.

3. Lógica e Modelagem

Desenvolver raciocínio lógico-matemático com proposições, operadores lógicos (E, OU, NÃO) e ordem de precedência.

4. Codificação na Linguagem C

Dominar a anatomia básica de programas em C, declaração de variáveis, tipos de dados primitivos e funções de Entrada/Saída (stdio.h).

Algoritmos transformam dados de entrada em resultados através de passos finitos

Definição e Propriedades de um Algoritmo

• Sequência Finita: Deve possuir um início claro e terminar após um número finito de etapas.

• Não Ambiguidades: Cada instrução deve ser precisa e ter interpretação única pelo agente executante.

• Entrada e Saída: Recebe dados iniciais (entradas) e produz um resultado correto e esperado (saída).

• Foco na Lógica: Exige a compreensão completa das regras do problema antes da escrita de código.

Etapas do Desenvolvimento da Solução

• 1. Entendimento do Problema

Compreensão clara dos requisitos, restrições e dados de entrada fornecidos.

• 2. Decomposição e Planejamento

Divisão do problema em sub-rotinas e estruturação do raciocínio lógico.

• 3. Construção do Algoritmo

Representação do fluxo lógico via pseudocódigo ou fluxograma.

• 4. Codificação na Linguagem C

Tradução formal das instruções para a sintaxe executável da linguagem C.

Pseudocódigo e fluxogramas formalizam a lógica antes da codificação formal

Formas Principais de Representação

• Pseudocódigo (Portugol): Representação textual estruturada próxima da linguagem humana. Permite focar na lógica sem se preocupar com detalhes sintáticos rígidos da máquina.

• Fluxogramas: Notação gráfica padronizada para mapear visualmente o fluxo de dados e tomadas de decisão utilizando formas geométricas universais:

• - OVAL: Início e Fim do processo.
- PARALELOGRAMO: Entrada/Saída de dados.
- RETÂNGULO: Processamento/Cálculo.
- LOSANGO: Decisão/Condição (Sim/Não).

Exemplo Prático: Algoritmo de Soma

```
Inicio
    Declare n1, n2, soma: Inteiro

    Escreva "Digite o primeiro número:"
    Leia n1

    Escreva "Digite o segundo número:"
    Leia n2

    soma <- n1 + n2

    Escreva "A soma é: ", soma
Fim
```

O pensamento computacional estrutura o raciocínio humano em 4 pilares essenciais

Pilar 1: DECOMPOSIÇÃO

Divisão de um problema complexo em subproblemas menores, mais fáceis de gerenciar e resolver isoladamente.

Pilar 2: RECONHECIMENTO DE PADRÕES

Identificação de semelhanças, tendências ou características repetitivas em problemas para reaproveitar soluções.

Pilar 3: ABSTRAÇÃO

Filtragem dos detalhes irrelevantes para focar estritamente nos aspectos essenciais do modelo computacional.

Pilar 4: ALGORITMOS

Criação do conjunto de instruções passo a passo, claras e finitas, para a automação da solução final.

A lógica proposicional e os operadores definem a tomada de decisão em código

p	q	p && q (E)	p || q (OU)
V	V	**V**	**V**
V	F	**F**	**V**
F	V	**F**	**V**
F	F	**F**	**F**

Modelagem Lógico-Matemática

Operadores Lógicos em C:

- && (AND): Verdadeiro apenas se ambas as proposições forem Verdadeiras.
- || (OR): Verdadeiro se pelo menos uma proposição for Verdadeira.
- ! (NOT): Inverte o valor lógico da proposição (V vira F e vice-versa).

Modelagem Matemática: Tradução de equações algébricas reais para expressões executáveis, respeitando estritamente as regras de precedência aritmética e lógica.

A Linguagem C une suporte a operações de baixo nível à estrutura de médio/alto nível

1. Linguagem de Máquina

Código binário direto (0s e 1s). Execução direta pelo hardware, porém extremamente complexo para humanos.

2. Assembly (Baixo Nível)

Uso de mnemonicos (MOV, ADD). Dependente de arquitetura de hardware específica.

3. Linguagem C (Estruturada)

★ Importância Histórica: O C é a 'mãe' das linguagens modernas. Sua sintaxe influenciou diretamente C++, Java, C# e JavaScript, tornando-se o padrão fundamental para o ensino de computação.

O compilador atua como um tradutor essencial entre o código humano e a máquina

Etapa 1
Código Fonte

Arquivo .c contendo o código de alto nível em C.

Etapa 2
Pré-Processador

Processa diretivas (#include, #define) e limpa comentários.

Etapa 3
Compilador

Traduz o código para linguagem Assembly / Objeto (.obj).

Etapa 4
Linker (Ligador)

Combina códigos objeto com bibliotecas do sistema.

Etapa 5
Executável

Arquivo binário executável (.exe ou nativo).

Papel do Compilador no Desenvolvimento:

Além da tradução, o compilador realiza verificação sintática e otimização lógica do código. Caso ocorram erros de sintaxe durante a compilação, o processo é interrompido antes de gerar o executável.

Ambientes Integrados (IDEs) combinam edição, compilação e depuração

O que é uma IDE e o Code::Blocks?

• Atenção Conceitual: O Code::Blocks NÃO É UM COMPILADOR. Ele é um Ambiente de Desenvolvimento Integrado (IDE) que gerencia ferramentas externas como o GCC/MinGW.

• Recursos de uma IDE: Editor de texto com realce sintático (coloração), gerenciamento de projetos, compilação em 1 clique e depuração integrada.

• Outras IDEs Famosas: Visual Studio, Dev-C++, Eclipse, CLion.

Conceito de Bug e Depuração (Debugging)

• O que é Bug? Uma falha ou erro de lógica no programa que produz comportamento inesperado, apesar do código ser sintaticamente aceito pelo compilador.

• Origem Histórica: Insetos reais (mariposas) que entravam em computadores valvulados e causavam curtos-circuitos elétricos.

• Depuração (Debugging): Processo sistemático de acompanhar a execução do programa linha por linha para identificar e tratar o erro.

A estrutura linear do C exige declaração de bibliotecas e função principal

Código Estruturado Mínimo em C

```
#include <stdio.h>

// Função principal de entrada
int main() {

    printf("Bem-vindo a APC!\n");

    return 0; // Finalização com sucesso
}
```

Análise dos Elementos da Sintaxe

- **1. Diretivas do Pré-processador**
  Instrução #include <stdio.h> importa funções de entrada e saída padrão.
- **2. Ponto de Entrada (main)**
  Todo programa em C inicia obrigatoriamente pela função int main().
- **3. Delimitadores { } e ;**
  Blocos de comando são delimitados por chaves e instruções finalizadas por ponto e vírgula ;.
- **4. Sensibilidade (Case-Sensitive)**
  C diferencia maiúsculas de minúsculas (main != Main, int != INT).

Tipos de dados primitivos definem o uso de memória RAM e a faixa de valores

Tipo Primitivo	Tamanho (Bits/Bytes)	Especificador (scanf/printf)	Intervalo de Valores	Exemplo de Uso
char	8 bits (1 Byte)	%c	-128 a 127 (ou 0 a 255)	char letra = 'A';
int	16/32 bits (2/4 Bytes)	%d ou %i	-32.768 a 32.767 (16 bits)	int idade = 20;
float	32 bits (4 Bytes)	%f	3,4E-38 a 3,4E+38	float altura = 1.75;
double	64 bits (8 Bytes)	%lf	1,7E-308 a 1,7E+308	double precisao = 3.141592;

Regras de Identificadores e Palavras Reservadas:

- Nomes de variáveis devem ser significativos (ex: saldo_conta em vez de s).
- Não podem começar com números nem conter espaços/caracteres especiais.
- Proibição de usar Palavras Reservadas do C (ex: int, float, return, if, else, for, struct).

As funções printf() e scanf() conectam a interação do usuário ao programa

Saída de Dados: printf()

Utilizado para exibir mensagens e valores formatados no terminal.

Sintaxe: printf("Texto %especificador", variavel);

Sequências de Escape Frequentes:
• \n : Quebra de linha (pula para a próxima linha).
• \t : Tabulação horizontal (espaçamento alinhado).

Exemplo:
printf("Idade: %d anos\n", idade);

Entrada de Dados: scanf()

Utilizado para ler dados digitados pelo usuário via teclado.

Sintaxe: scanf("%especificador", &variavel);

REGRA DE OURO (&):
O operador & é OBRIGATÓRIO no scanf() para passar o endereço de memória onde o valor será armazenado!

Exemplo:
scanf("%f", &altura);

O exemplo integrador de área do círculo consolida a estrutura completa em C

Calculadora de Geometria Circular em C

```c
#include <stdio.h>
#define PI 3.14159265

int main() {
    float raio, area, perimetro;

    printf("Informe o valor do raio: ");
    scanf("%f", &raio); // Operador & obrigatorio

    // Processamento Matematico
    area = PI * (raio * raio);
    perimetro = 2 * PI * raio;

    // Saida Formatada
    printf("\n--- RESULTADOS ---\n");
    printf("Area calculada: %.4f\n", area);
    printf("Perimetro     : %.4f\n", perimetro);

    return 0;
}
```

Destaques do Código

- #define PI: Declaração de constante simbólica via pré-processador.
- float: Escolha do tipo de dado de ponto flutuante para tratar casas decimais.
- %.4f: Formatador de saída que limita a exibição a 4 casas decimais.
- Entrada/Saída: Uso combinado de printf() para orientação ao usuário e scanf() para leitura.

Boas práticas de código garantem legibilidade, manutenção e evitam bugs comuns

Boas Práticas de Programação

• Indentação Rígida: Uso de tabulações para destacar a hierarquia dos blocos de comando.

• Comentários Clareadores: Uso de // (linha única) ou /* ... */ (múltiplas linhas) para documentar a lógica.

• Princípios DRY & KISS: 'Don't Repeat Yourself' (evitar duplicação) e 'Keep It Simple' (manter a simplicidade).

Erros Frequentes para Evitar

• 1. Esquecer o Ponto e Vírgula (;): Causa erro sintático imediato de compilação na linha seguinte.

• 2. Omitir o Operador & no scanf(): Provoca falha de segmentação (Segmentation Fault / crash) ao tentar escrever em endereço de memória inválido.

• 3. Especificadores Incorretos: Usar %d para ler variáveis float (gera corrupção de valores na memória).

• Próximos Passos (Unidade II): Estruturas Condicionais (if-else, switch) e Laços de Repetição (while, for).

UDF
Centro Universitário

ALGORITMOS E
PENSAMENTO COMPUTACIONAL
2026.2

UNIDADE I

Fundamentos da Introdução à Programação e ao Pensamento Computacional em C

Prof. Me. Gleidson Porto Batista

UDF – Centro Universitário



ALGORITMO · LÓGICA · PROBLEMAS · MODELAGEM · PROGRAMAÇÃO · SOLUÇÕES

Seu futuro,
nossa inspiração.

CIÊNCIA DA COMPUTAÇÃO
SISTEMAS DE INFORMAÇÃO | ENGENHARIA DE SOFTWARE